

R-Control IP4 – Modbus TCP Interface

This document describes the **Modbus TCP slave** interface of the R-Control IP4 from an integration perspective.

Status: Implementation in `components/modbus_server` (firmware repo). The mapping is designed for up to **4 relays** and **4 inputs**.

1. Overview

Purpose

- Switch relays via coils.
- Read input/relay status as well as NTC telemetry via discrete inputs and input registers.

Characteristics

- Protocol: Modbus TCP (slave)
- Default port: `502`
- Unit ID: configurable (default `1`)
- Network binding: binds (if available) to the Ethernet netif `ETH_DEF` (otherwise without explicit netif binding).

2. Activation & Configuration

Settings are located in the configuration under:

- `config.interfaces.modbus_tcp.enabled` (bool)
- `config.interfaces.modbus_tcp.port` (int, `1..65535`, default `502`)
- `config.interfaces.modbus_tcp.unit_id` (int, `1..247`, default `1`)

In the web UI: **Interfaces** → **Modbus TCP Settings**.

3. Protocol Parameters & Limits

Data areas (firmware-internal)

- Coils: 8 bits (of which `0..3` are used; `4..7` currently unused)
- Discrete inputs: 16 bits
- Holding registers: 16 registers (currently reserved)
- Input registers: 80 registers

Update / cycle

- Coil write accesses are applied to the relays cyclically (approx. every 100 ms).

4. Addressing & Data Types

Important notes for Modbus clients:

- In this document, addresses are given as **0-based offsets**.
- Many tools (SCADA/HMI) show **1-based** addresses. Then often: offset **N** → displayed as **N+1**.
- Register endianness:
 - Modbus is big-endian per 16-bit register.
 - For 32-bit values, the firmware uses **low word first, high word second**:
 - `value32 = lo16 | (hi16 << 16)`
- Signed/unsigned:
 - Temperature is stored in a 16-bit register as **int16 (°C×100)**. Clients must interpret the register as signed.

5. Register Map

Note: All addresses/offsets are **0-based**.

5.1 Overview (all areas)

Area	FC	Offset / Range	Type	Access	Meaning
Coils	01/05/15	0..3	bit	R/W	Relays (R1..R4) desired state (applied cyclically)
Discrete Inputs	02	0..3	bit	R	Inputs (S1..S4) pressed
Discrete Inputs	02	4..7	bit	R	Relays (R1..R4) actual state
Discrete Inputs	02	8..11	bit	R	Inputs (S1..S4) detached
Discrete Inputs	02	12..15	bit	R	Inputs (S1..S4) enabled
Input Registers	04	0	int16 ×0.01	R	Temperature °C×100
Input Registers	04	1	uint16	R	NTC status (<code>ntc_sensor_status_t</code>)
Input Registers	04	2	uint16	R	NTC mV
Input Registers	04	3	uint16	R	NTC raw ADC
Input Registers	04	10..13	uint16	R	pressed (S1..S4) as 0/1
Input Registers	04	20..23	uint16	R	last event id (S1..S4, <code>BUTTON_EVENT_*</code>)

Area	FC	Offset / Range	Type	Access	Meaning
Input Registers	04	30..37	uint16	R	event counter (S1..S4), 2 registers each: lo/hi (32-bit)
Input Registers	04	40..47	uint16	R	last timestamp ms (S1..S4), 2 registers each: lo/hi (32-bit)
Input Registers	04	50..53	uint16	R	input enabled (S1..S4) as 0/1
Input Registers	04	54..57	uint16	R	input detached (S1..S4) as 0/1
Input Registers	04	60..63	uint16	R	relay state (R1..R4) as 0/1
Holding Registers	03/06/16	0..15	uint16	R/W	reserved (currently without semantics)

Coils (FC 01/05/15) – Switching Relays

Start offset: 0

Offset	Meaning	Access
0	Relay 0 (R1) desired state (0/1)	R/W
1	Relay 1 (R2) desired state (0/1)	R/W
2	Relay 2 (R3) desired state (0/1)	R/W
3	Relay 3 (R4) desired state (0/1)	R/W

Behavior: Coil writes are applied to the relays cyclically (approx. every 100 ms).

Discrete Inputs (FC 02) – Status Bits (read-only)

Start offset: 0

Bit	Meaning
0..3	Input pressed (S1..S4): 1 = pressed
4..7	Relay state (R1..R4): 1 = relay is ON
8..11	Input detached (S1..S4): 1 = detached
12..15	Input enabled (S1..S4): 1 = channel enabled

Input Registers (FC 04) – Telemetry & Event Info (read-only)

Start offset: 0

Temperature / NTC Snapshot

Offset	Type	Meaning
0	int16 (x100)	Temperature in °C × 100 (e.g. 2345 = 23.45 °C). On error status, the value may be 0.
1	uint16	NTC status (ntc_sensor_status_t).
2	uint16	NTC reading in mV (unclamped to 0..65535).
3	uint16	NTC raw ADC (unclamped to 0..65535).

Inputs: pressed / last event / counter / timestamp

For each input *i* (0..3):

Offset	Type	Meaning
10 + i	uint16	pressed: 1 = pressed, 0 = not pressed
20 + i	uint16	last event id (matches BUTTON_EVENT_* ID)
30 + 2*i	uint16	event counter low word
30 + 2*i + 1	uint16	event counter high word
40 + 2*i	uint16	last timestamp low word (ms)
40 + 2*i + 1	uint16	last timestamp high word (ms)

32-bit reconstruction:

- event_count_32 = cnt_lo | (cnt_hi << 16)
- timestamp_ms_32 = ts_lo | (ts_hi << 16)

Configuration / Status Mirrors

Offset	Type	Meaning
50 + i	uint16	input enabled (S1..S4): 1/0
54 + i	uint16	input detached (S1..S4): 1/0
60 + i	uint16	relay state (R1..R4): 1/0

Holding Registers (FC 03/06/16)

Currently reserved, but in the firmware still without defined semantics.

6. Integration Notes (Best Practices)

- Relay actual state is provided redundantly:
 - Discrete inputs bits 4..7

- Input registers **60..63** (each 0/1)
- **Coils** are the write channel; the firmware applies coil states cyclically to the physical relays (approx. 100 ms).
- **Avoid toggle**: Writing coils is idempotent (0/1). For deterministic control, integrators should generally use **0/1** instead of "toggle" logic.
- **"detached"** means: input is decoupled from the relay coupling; events/status still exist.

7. Security Notes

Modbus TCP is **not authenticated** in this firmware.

Recommended measures:

- Operate only in a trusted LAN/OT network.
- Restrict access via firewall/VLAN to defined source IPs.
- If using a Modbus gateway/proxy: enforce authentication/ACLs/TLS there.

8. Examples

8.1 Example: Read temperature

- FC04, offset **0**, length **2** (temperature + NTC status)

8.2 Example: Turn relay 1 on

- FC05 (Write Single Coil), offset **0**, value **1**

8.3 Example: Read relay state

- FC02 (Read Discrete Inputs), offset **4**, length **4** (bits 4..7)

9. Troubleshooting

- **No connection**: check port/firewall (`config.interfaces.modbus_tcp.port`, default **502**).
- **Wrong offsets**: check whether the client uses 0-based or 1-based addressing.
- **Unit ID mismatch**: check `config.interfaces.modbus_tcp.unit_id` (1..247).
- **Implausible temperature**: interpret register as `int16` (signed) and apply scaling $\times 0.01$.

10. Home Assistant Example (configuration.yaml)

The following example configuration integrates R-Control IP4 via **Modbus TCP** and creates:

- **4 coil switches** (Modbus coils) as a "write channel" for the relays
- **4 relay-state binary sensors** (FC02 bits 4..7) as the relays' actual state
- **4 template switches (synced)** that show the actual state but switch via coils
- Temperature/NTC readings as `sensor` (input registers)
- Input status (pressed/enabled/detached) as `binary_sensor`
- Button event ID, counter, and timestamp as `sensor` + template combination (2×16-bit → 32-bit)

Important:

- The offsets documented here are **0-based**.
- Home Assistant typically also uses **0-based** register addresses for the Modbus integration backend. If you see it "shifted by 1" in the UI, check the documentation of your client/frontend.

```
modbus:
- name: rcontrol_ip4
  type: tcp
  host: 192.168.1.50 # <-- device IP
  port: 502
  # Optional: delay between requests
  # delay: 0

# Relays via coils (FC01/05) - write channel only
# (Actual state comes separately via discrete inputs bits 4..7)
switches:
- name: "rcontrol_relay1_coil"
  slave: 1
  address: 0
  write_type: coil
- name: "rcontrol_relay2_coil"
  slave: 1
  address: 1
  write_type: coil
- name: "rcontrol_relay3_coil"
  slave: 1
  address: 2
  write_type: coil
- name: "rcontrol_relay4_coil"
  slave: 1
  address: 3
  write_type: coil

# Discrete inputs (FC02)
binary_sensors:
  # Inputs pressed (bits 0..3)
- name: "rcontrol_s1_pressed"
  slave: 1
  address: 0
  input_type: discrete_input
- name: "rcontrol_s2_pressed"
  slave: 1
  address: 1
  input_type: discrete_input
- name: "rcontrol_s3_pressed"
  slave: 1
  address: 2
  input_type: discrete_input
- name: "rcontrol_s4_pressed"
  slave: 1
  address: 3
  input_type: discrete_input
```

```
# Relay actual state (bits 4..7)
- name: "rcontrol_relay1_state"
  slave: 1
  address: 4
  input_type: discrete_input
- name: "rcontrol_relay2_state"
  slave: 1
  address: 5
  input_type: discrete_input
- name: "rcontrol_relay3_state"
  slave: 1
  address: 6
  input_type: discrete_input
- name: "rcontrol_relay4_state"
  slave: 1
  address: 7
  input_type: discrete_input

# detached (bits 8..11)
- name: "rcontrol_s1_detached"
  slave: 1
  address: 8
  input_type: discrete_input
- name: "rcontrol_s2_detached"
  slave: 1
  address: 9
  input_type: discrete_input
- name: "rcontrol_s3_detached"
  slave: 1
  address: 10
  input_type: discrete_input
- name: "rcontrol_s4_detached"
  slave: 1
  address: 11
  input_type: discrete_input

# enabled (bits 12..15)
- name: "rcontrol_s1_enabled"
  slave: 1
  address: 12
  input_type: discrete_input
- name: "rcontrol_s2_enabled"
  slave: 1
  address: 13
  input_type: discrete_input
- name: "rcontrol_s3_enabled"
  slave: 1
  address: 14
  input_type: discrete_input
- name: "rcontrol_s4_enabled"
  slave: 1
  address: 15
  input_type: discrete_input
```

```
# Input registers (FC04)
sensors:
- name: "rcontrol_ntc_temperature"
  slave: 1
  address: 0
  input_type: input
  data_type: int16
  scale: 0.01
  precision: 2
  unit_of_measurement: "°C"
  device_class: temperature

- name: "rcontrol_ntc_status"
  slave: 1
  address: 1
  input_type: input
  data_type: uint16

- name: "rcontrol_ntc_mv"
  slave: 1
  address: 2
  input_type: input
  data_type: uint16
  unit_of_measurement: "mV"

- name: "rcontrol_ntc_raw_adc"
  slave: 1
  address: 3
  input_type: input
  data_type: uint16

# last event id (20..23)
- name: "rcontrol_s1_last_event_id"
  slave: 1
  address: 20
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_last_event_id"
  slave: 1
  address: 21
  input_type: input
  data_type: uint16
- name: "rcontrol_s3_last_event_id"
  slave: 1
  address: 22
  input_type: input
  data_type: uint16
- name: "rcontrol_s4_last_event_id"
  slave: 1
  address: 23
  input_type: input
  data_type: uint16

# Event counter (low/high) and timestamp (low/high)
```



```
# S1
- name: "rcontrol_s1_event_cnt_lo"
  slave: 1
  address: 30
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_event_cnt_hi"
  slave: 1
  address: 31
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_ts_ms_lo"
  slave: 1
  address: 40
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_ts_ms_hi"
  slave: 1
  address: 41
  input_type: input
  data_type: uint16

# S2
- name: "rcontrol_s2_event_cnt_lo"
  slave: 1
  address: 32
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_event_cnt_hi"
  slave: 1
  address: 33
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_ts_ms_lo"
  slave: 1
  address: 42
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_ts_ms_hi"
  slave: 1
  address: 43
  input_type: input
  data_type: uint16

# S3
- name: "rcontrol_s3_event_cnt_lo"
  slave: 1
  address: 34
  input_type: input
  data_type: uint16
- name: "rcontrol_s3_event_cnt_hi"
  slave: 1
  address: 35
  input_type: input
```

```

    data_type: uint16
  - name: "rcontrol_s3_ts_ms_lo"
    slave: 1
    address: 44
    input_type: input
    data_type: uint16
  - name: "rcontrol_s3_ts_ms_hi"
    slave: 1
    address: 45
    input_type: input
    data_type: uint16

# S4
  - name: "rcontrol_s4_event_cnt_lo"
    slave: 1
    address: 36
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_event_cnt_hi"
    slave: 1
    address: 37
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_ts_ms_lo"
    slave: 1
    address: 46
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_ts_ms_hi"
    slave: 1
    address: 47
    input_type: input
    data_type: uint16

```

template:

```

  - sensor:
    - name: "rcontrol_s1_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s1_event_cnt_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s1_event_cnt_lo')|int(0)) }}
      icon: mdi:counter
    - name: "rcontrol_s1_last_timestamp_ms"
      state: >-
        {{ (states('sensor.rcontrol_s1_ts_ms_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s1_ts_ms_lo')|int(0)) }}
      icon: mdi:clock-outline

    - name: "rcontrol_s2_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s2_event_cnt_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s2_event_cnt_lo')|int(0)) }}
      icon: mdi:counter
    - name: "rcontrol_s2_last_timestamp_ms"
      state: >-

```

```

        {{ (states('sensor.rcontrol_s2_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s2_ts_ms_lo')|int(0)) }}
        icon: mdi:clock-outline

    - name: "rcontrol_s3_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s3_event_cnt_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s3_event_cnt_lo')|int(0)) }}
        icon: mdi:counter
    - name: "rcontrol_s3_last_timestamp_ms"
      state: >-
        {{ (states('sensor.rcontrol_s3_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s3_ts_ms_lo')|int(0)) }}
        icon: mdi:clock-outline

    - name: "rcontrol_s4_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s4_event_cnt_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s4_event_cnt_lo')|int(0)) }}
        icon: mdi:counter
    - name: "rcontrol_s4_last_timestamp_ms"
      state: >-
        {{ (states('sensor.rcontrol_s4_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s4_ts_ms_lo')|int(0)) }}
        icon: mdi:clock-outline

  - switch:
    # "Synced" switches: show the real actual state, but switch via the coil
    switches.
    # Note: If you change the `name:` values, the entity IDs will also change.
    # Then the referenced `entity_id:` below must be adjusted accordingly.
    - name: "rcontrol_relay1"
      state: "{{ is_state('binary_sensor.rcontrol_relay1_state', 'on') }}"
      turn_on:
        - service: switch.turn_on
          target:
            entity_id: switch.rcontrol_relay1_coil
      turn_off:
        - service: switch.turn_off
          target:
            entity_id: switch.rcontrol_relay1_coil

    - name: "rcontrol_relay2"
      state: "{{ is_state('binary_sensor.rcontrol_relay2_state', 'on') }}"
      turn_on:
        - service: switch.turn_on
          target:
            entity_id: switch.rcontrol_relay2_coil
      turn_off:
        - service: switch.turn_off
          target:
            entity_id: switch.rcontrol_relay2_coil

    - name: "rcontrol_relay3"

```

```

state: "{{ is_state('binary_sensor.rcontrol_relay3_state', 'on') }}"
turn_on:
  - service: switch.turn_on
    target:
      entity_id: switch.rcontrol_relay3_coil
turn_off:
  - service: switch.turn_off
    target:
      entity_id: switch.rcontrol_relay3_coil

- name: "rcontrol_relay4"
  state: "{{ is_state('binary_sensor.rcontrol_relay4_state', 'on') }}"
  turn_on:
    - service: switch.turn_on
      target:
        entity_id: switch.rcontrol_relay4_coil
  turn_off:
    - service: switch.turn_off
      target:
        entity_id: switch.rcontrol_relay4_coil

```

Home Assistant: Showing relay state correctly (on local changes)

If a relay is switched **locally on the device** (button/GUI/automation), you want to see the **actual state** in Home Assistant.

This firmware provides the actual state redundantly:

- **Discrete inputs bits 4..7**: relay state (recommended for HA `binary_sensor`)
- **Input registers 60..63**: relay state as 0/1

Recommendation:

- Use the **template switches (syncd)** in the UI (`switch.rcontrol_relay1..4`).
 - Display: real actual state from `binary_sensor.rcontrol_relayX_state`
 - Switching: via `switch.rcontrol_relayX_coil` (Modbus coil)

This keeps Home Assistant properly synchronized, even if the state changes locally on the device.

Optional:

- For S2..S4 use the corresponding offsets:
 - Event counter: $30 + 2*i$ (lo) / $30 + 2*i + 1$ (hi)
 - Timestamp: $40 + 2*i$ (lo) / $40 + 2*i + 1$ (hi)